

Lossy & Lossless Decomposition in Database Design

August 11, 2025

Introduction

In relational database design, large relations are often decomposed into smaller ones to:

- Remove redundancy
- Avoid anomalies
- Improve maintainability

However, not all decompositions are safe. Two main types exist:

- **Lossy Decomposition** – Information loss or spurious tuples appear.
- **Lossless Decomposition** – No information is lost; original relation can be perfectly reconstructed.

Lossy Decomposition

A decomposition of a relation R into $R_1, R_2, \dots, R_n$ is **lossy** if:

$$R \neq R_1 \bowtie R_2 \bowtie \dots \bowtie R_n$$

This means joining the decomposed tables does not exactly reproduce the original table — *extra tuples (spurious tuples)* or missing tuples appear.

Lossless Decomposition

A decomposition is **lossless** if:

$$R = R_1 \bowtie R_2 \bowtie \cdots \bowtie R_n$$

The original relation can be reconstructed exactly without introducing any spurious tuples or losing data.

Lossy Decomposition Example

Original Relation

$R(\text{StudentID}, \text{Course}, \text{Instructor})$

StudentID	Course	Instructor
101	DBMS	John
101	AI	Smith
102	DBMS	John

Decomposition

$R_1(\text{StudentID}, \text{Course}) \quad R_2(\text{Course}, \text{Instructor})$

R_1 :

StudentID	Course
101	DBMS
101	AI
102	DBMS

R_2 :

Course	Instructor
DBMS	John
AI	Smith

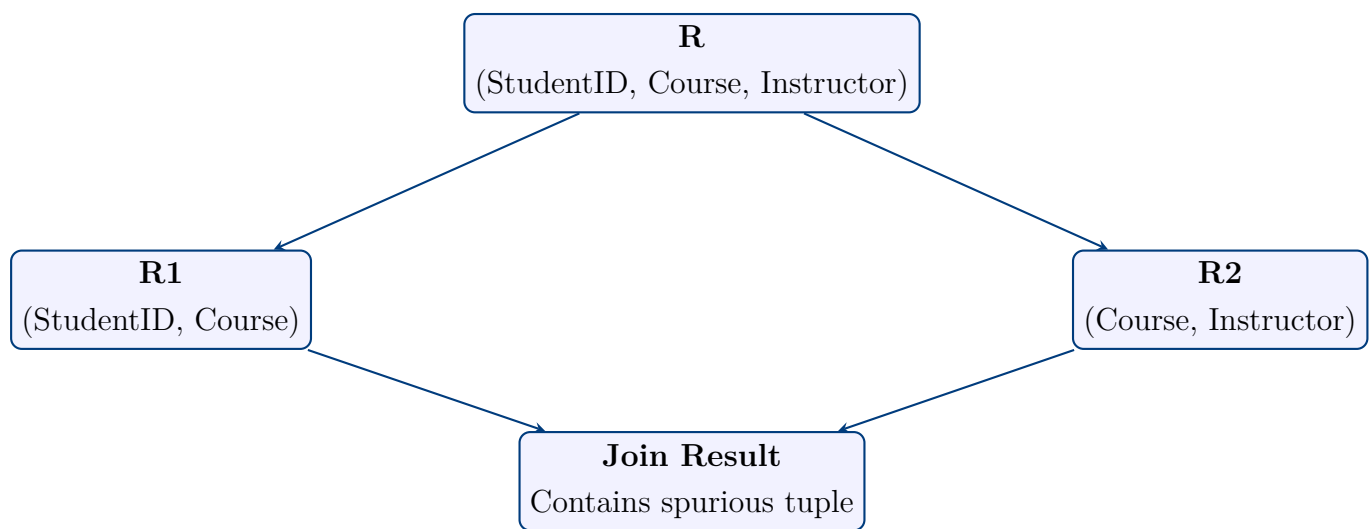
Join Result (Problem)

Joining R_1 and R_2 on Course:

StudentID	Course	Instructor
101	DBMS	John
101	AI	Smith
102	DBMS	John
102	AI	Smith

(102, AI, Smith) did not exist in the original — this is **Lossy Decomposition**.

Visual Representation Lossy Decomposition



Lossless Decomposition Example

Original Relation

$R(\text{StudentID}, \text{Name}, \text{Department})$

StudentID	Name	Department
101	Alice	CS
102	Bob	IT
103	Carol	CS

Decomposition (Lossless)

$R_1(\text{StudentID}, \text{Name})$ $R_2(\text{StudentID}, \text{Department})$

R_1 :

StudentID	Name
101	Alice
102	Bob
103	Carol

R_2 :

StudentID	Department
101	CS
102	IT
103	CS

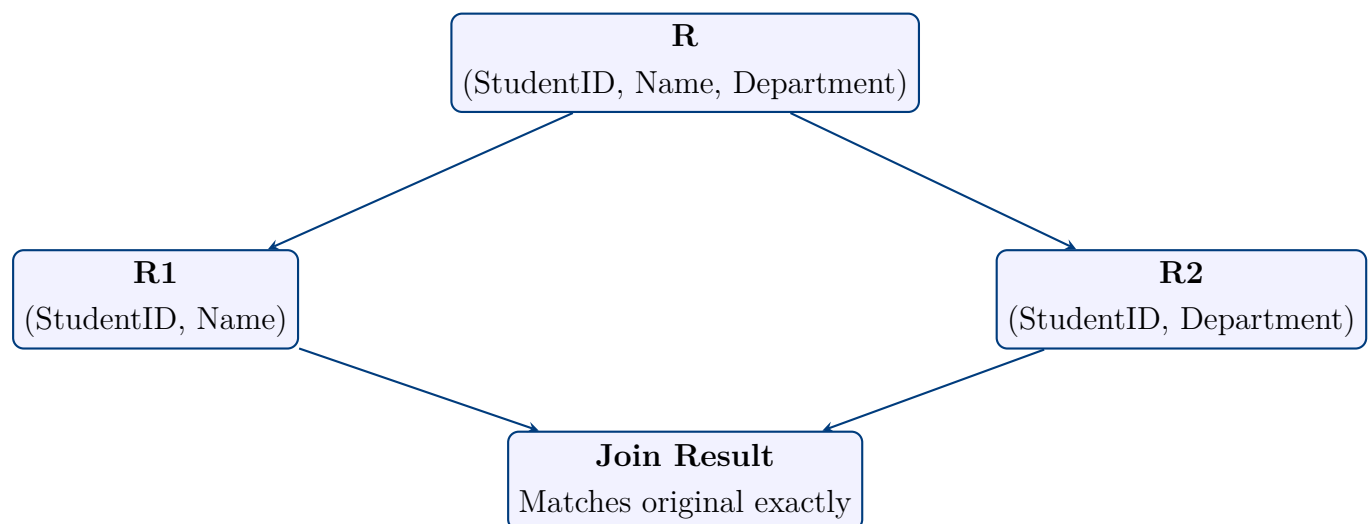
Join Result (No Problem)

Joining R_1 and R_2 on StudentID:

StudentID	Name	Department
101	Alice	CS
102	Bob	IT
103	Carol	CS

Here, the join result exactly matches the original relation — **Lossless Decomposition**.

Visual Representation of Lossless Decomposition



Visual Comparison

Property	Lossy	Lossless
Data Reconstruction	Original data not exactly reproduced; spurious tuples possible	Original data reproduced exactly
Accuracy	Incorrect query results possible	Always correct
Condition	Common attributes are <u>not</u> a key in any sub-relation	Common attributes form a key in at least one sub-relation
Impact	Leads to anomalies	Safe decomposition

Formal Condition for Lossless Join

Definition: Lossless-Join Decomposition

A decomposition of a relation R into R_1 and R_2 is **lossless** if:

$$(R_1 \cap R_2) \rightarrow R_1 \quad \text{or} \quad (R_1 \cap R_2) \rightarrow R_2$$

That is, the **common attributes** (intersection) must be a **key** for at least one of the decomposed relations.

Explanation

Let R be a relation schema, and F be a set of functional dependencies on R . Suppose we decompose R into two relations R_1 and R_2 .

For a lossless-join decomposition, it must be true that for every relation r over R :

$$r = \pi_{R_1}(r) \bowtie \pi_{R_2}(r)$$

(where π denotes projection and $\bowtie$ is the natural join operator).

In SQL terms:

```
SELECT * FROM r =  
SELECT * FROM (SELECT R1 FROM r) NATURAL JOIN (SELECT R2 FROM r)
```

The above means that no data is lost when decomposing and then joining the tables.

Sufficient Condition for Lossless Join

The decomposition of R into R_1 and R_2 is lossless if at least one of the following holds in the closure F^+ :

$$R_1 \cap R_2 \rightarrow R_1 \quad \text{or} \quad R_1 \cap R_2 \rightarrow R_2$$

Note: These dependencies form a sufficient condition for lossless join. They are necessary if all constraints are functional dependencies.

Impact of Nullable Attributes on Left-Hand Side of Functional Dependencies

Issue with Nullable LHS Attributes

In lossless decomposition theory, it is assumed that attributes on the **left-hand side (LHS)** of a functional dependency (FD) cannot be **null**. Violating this assumption may cause serious problems:

- **Ambiguity in FD enforcement:** When LHS attributes contain NULL, it becomes unclear whether two tuples have the same LHS value, because in SQL semantics, $\text{NULL} = \text{NULL}$ evaluates to **FALSE**.
- **Loss of losslessness:** Decomposition may no longer be lossless. Joining decomposed relations may generate **spurious tuples** or fail to reconstruct the original tuples correctly.
- **Breakdown of FD semantics:** The FD $X \rightarrow Y$ means tuples agreeing on X must agree on Y . If X contains nulls, this guarantee cannot be enforced reliably, since equality comparisons on nulls are undefined.

Intuition: A functional dependency requires a *definite* key value to determine dependent attributes uniquely. If the key is unknown or missing (null), the dependency check becomes unreliable, causing errors in reconstruction after decomposition.

Aspect	Consequence if LHS Can Be Null
FD Checking	Ambiguous; NULL values make equality comparison impossible
Lossless Decomposition	May become lossy; spurious tuples or loss of tuples after join
Data Integrity	FD semantics no longer guarantee uniqueness of dependent attributes

Examples

Example: Lossless Decomposition Check

Lossless Decomposition Example

Consider the relation schema:

$$R = (A, B, C)$$

with functional dependencies:

$$F = \{A \rightarrow B, \quad B \rightarrow C\}.$$

Case 1:

$$R_1 = (A, B), \quad R_2 = (B, C)$$

Common attributes:

$$R_1 \cap R_2 = \{B\}.$$

Since $B \rightarrow C$, we have:

$$B \rightarrow BC,$$

so B is a key for R_2 , and the decomposition is **lossless**.

Case 2:

$$R_1 = (A, B), \quad R_2 = (A, C)$$

Common attributes:

$$R_1 \cap R_2 = \{A\}.$$

Since $A \rightarrow B$, we have:

$$A \rightarrow AB,$$

so A is a key for R_1 , and the decomposition is **lossless**.

Note: This decomposition is **not dependency preserving** since the dependency $B \rightarrow C$ is lost and can only be enforced by joining R_1 and R_2 .

Example: Lossless Decomposition with Larger Schema

Lossless Decomposition Example with FDs

Suppose we decompose the schema:

$$R = (A, B, C, D, E)$$

into

$$R_1 = (A, B, C), \quad R_2 = (A, D, E),$$

with functional dependencies:

$$F = \{A \rightarrow BC, \quad CD \rightarrow E, \quad B \rightarrow D, \quad E \rightarrow A\}.$$

Lossless Decomposition Test: A decomposition R_1, R_2 of R is lossless if:

$$(R_1 \cap R_2) \rightarrow R_1 \quad \text{or} \quad (R_1 \cap R_2) \rightarrow R_2.$$

Step 1: Find common attributes

$$R_1 \cap R_2 = \{A\}.$$

Step 2: Compute attribute closure A^+ under F

$$\begin{aligned} A &\xrightarrow{A \rightarrow BC} A, B, C, \\ &\xrightarrow{B \rightarrow D} A, B, C, D, \\ &\xrightarrow{CD \rightarrow E} A, B, C, D, E. \end{aligned}$$

Thus,

$$A^+ = \{A, B, C, D, E\} = R.$$

Step 3: Conclusion Since A functionally determines all attributes of R , it determines all attributes of both R_1 and R_2 . Therefore, the decomposition is **lossless**.

□

Example: Lossless Decomposition Test for Given Decomposition

Lossless Decomposition Test with FDs

Suppose we decompose the schema:

$$R = (A, B, C, D, E)$$

into

$$R_1 = (A, B, C), \quad R_2 = (C, D, E),$$

with functional dependencies:

$$F = \{A \rightarrow BC, \quad CD \rightarrow E, \quad B \rightarrow D, \quad E \rightarrow A\}.$$

Lossless Decomposition Test: A decomposition R_1, R_2 of R is lossless if:

$$(R_1 \cap R_2) \rightarrow R_1 \quad \text{or} \quad (R_1 \cap R_2) \rightarrow R_2.$$

Step 1: Find common attributes

$$R_1 \cap R_2 = \{C\}.$$

Step 2: Compute attribute closure C^+ under F

$$C^+ = \{C\}.$$

No FD in F or its closure allows us to add any other attribute to C^+ .

Step 3: Conclusion Since C does not functionally determine all attributes of either R_1 or R_2 , the decomposition is **not lossless**.

□

Example: Multiple Distinct Lossless BCNF Decompositions

Multiple Distinct Lossless BCNF Decompositions

Consider the relation schema:

$$R' = (A, B, C)$$

with the set of functional dependencies:

$$F' = \{A \rightarrow B, \quad B \rightarrow C\}$$

Candidate Key: Since $A \rightarrow B$ and $B \rightarrow C$, by transitivity $A \rightarrow C$. Hence, A is a candidate key for R' .

Check BCNF:

- $A \rightarrow B$: Left side A is a candidate key, so this FD satisfies BCNF.
- $B \rightarrow C$: Left side B is not a candidate key, so violates BCNF.

Possible BCNF Decompositions:

1. Decompose using $B \rightarrow C$:

$$R_1 = (B, C), \quad R_2 = (A, B)$$

Both are in BCNF:

- R_1 — $B \rightarrow C$ with B as key.
- R_2 — $A \rightarrow B$ with A as key.

2. Decompose using $A \rightarrow B$ first, then check $B \rightarrow C$:

$$R_1 = (A, B), \quad R_2 = (B, C)$$

(Same as above, order reversed — distinct decomposition order.)

3. Decompose using $B \rightarrow C$ and then further decompose R_2 by trivial FD:

$$R_1 = (B, C), \quad R_2 = (A, B), \quad R_3 = (A, C)$$

(Though R_3 may be trivial or unnecessary, it forms a different decomposition set.)

All decompositions are lossless because:

- The intersection $R_1 \cap R_2 = \{B\}$ is a key in R_1 (or R_2) ensuring lossless join.
- Keys are preserved, and decompositions eliminate BCNF violations.

Conclusion: The relation $R' = (A, B, C)$ with $F' = \{A \rightarrow B, B \rightarrow C\}$ admits at least three distinct lossless BCNF decompositions, illustrating that multiple lossless decompositions can exist.

□